



Digital Implementation of Oscillatory Neural Network on FPGA

Prof. Sandip Lashkare
Chinmay Kulkarni

Why ONN?

- AI increasingly relies on neural networks like DNNs and CNNs.
- These models demand high memory, power, and compute due to Von Neumann architectures.
- Von Neumann systems face memory bottlenecks, limiting edge performance.

Non-Boolean Computation

CMOS Bottlenecks:
1. Power Density
2. Memory – Logic divide
3. NP – hard problems

Neuromorphic Architectures:
1. Spiking Neural Networks
2. Networks of Oscillators

fig 1: advantages of neuromorphic architectures

- Inspired by the brain, Neuromorphic computing offers lower power and memory needs.
- Oscillatory Neural Networks (ONNs) use phase synchronisation for efficient neuromorphic computing.
- ONNs enable fast parallel processing, robust associative memory, and ultra-low power use.

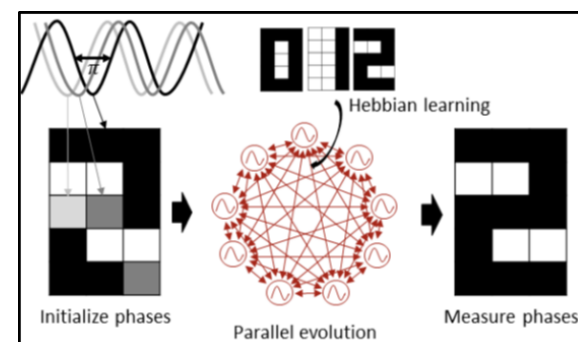
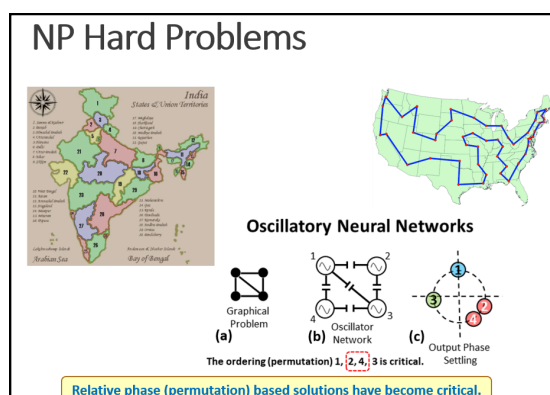
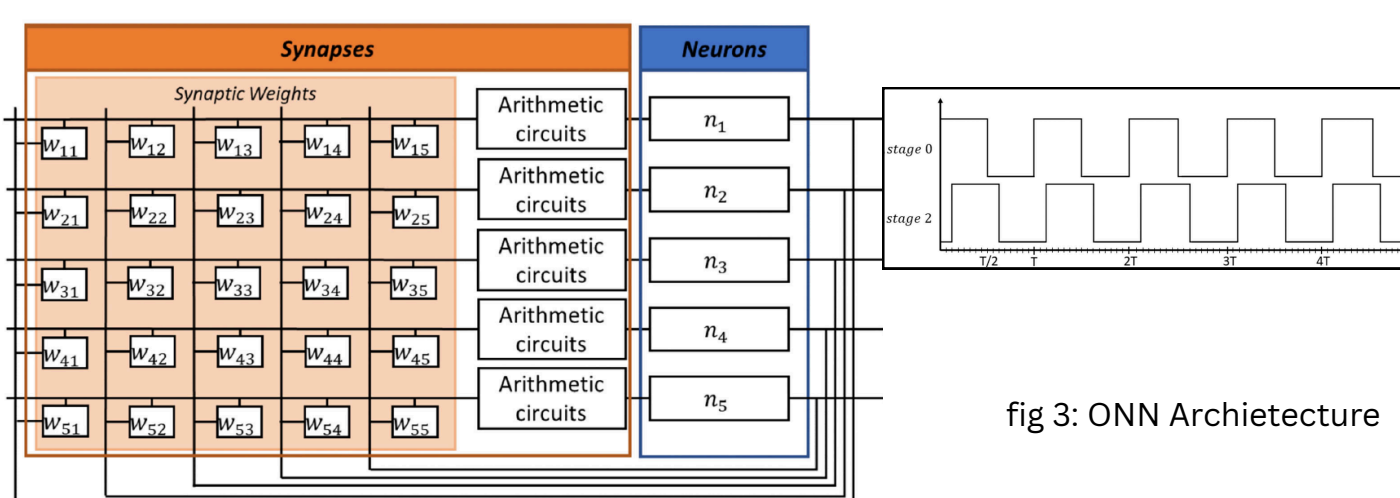


fig 2:- ONN for pattern recognition

What is ONN?

- A digital oscillatory neural network (ONN) was implemented on an FPGA, inspired by neuromorphic principles, where a 16-bit square wave encodes grayscale pixel information through its phase.
- These phase-encoded signals propagate through synapses, multiplied by synaptic weights and summed to generate new oscillatory signals according to the ONN update equations.



- The synaptic weights were trained offline using the Hebbian learning rule implemented in Python, then hardcoded into the FPGA design for inference.

$$w_{ij} = \sum_k \xi_i^k \xi_j^{kT} \quad \text{with } w_{ij} = 0 \quad \forall i = j.$$

- Each neuron calculates the phase difference ($\Delta\phi$) between its input phase (ϕ_{in}) and its output phase (ϕ_{out}), using this to update its output phase to achieve synchronisation with its input.
- The ONN synapse block holds the trained weights and computes each neuron's input oscillation, using an arithmetic logic unit to generate the summed input for the i -th neuron.

$$n_{in}[i] = \text{sign}(\sum_j w_{ij} - \sum_k w_{ik}) \quad \text{where } j \text{ extends to those neurons with } n_{out}[j]=1 \text{ and } k \text{ to those with } n_{out}[k]=0.$$

- After several iterations, the phases of all oscillators converge to a stable pattern that represents the output digit image.
- Digit classification is performed by measuring the Hamming distance between the converged output pattern and stored reference patterns, selecting the closest match as the recognised digit.

Recognition of Custom Digit Patterns

- Designed a digital ONN architecture for images of size 10×6 pixels.
- Created a custom dataset containing digit patterns from 0 to 9 in 10×6 resolution.



fig 4: digits of size 10X6

- Successfully achieved convergence of the network phases to the original digit patterns.
- Recognized digits using the Hamming Distance method for pattern matching.

Handwritten Digit Recognition

- Working with custom-designed digits is not sufficient; real handwritten digits must also be tested for practical validation.
- The MNIST dataset was used to recognise handwritten digits, originally in 28×28 resolution and resized to 10×10 pixels for the ONN.

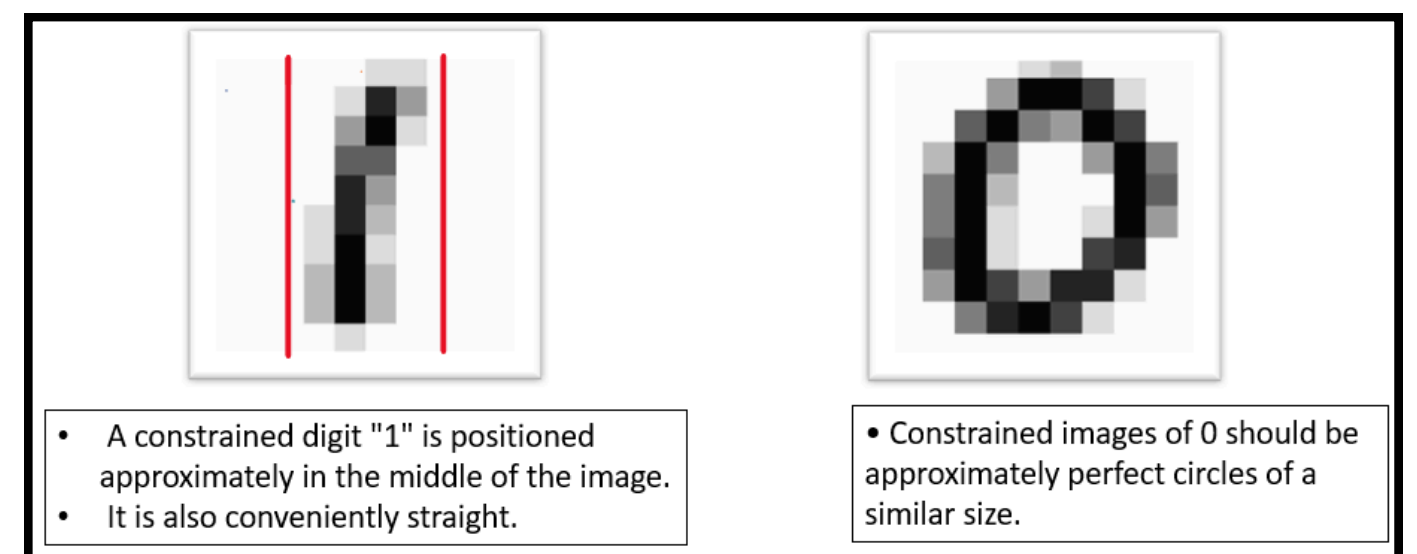


fig 5: constrained images

- Training was performed on constrained images for two digits (0 and 1) to evaluate the system.
- The network showed convergence close to the stored patterns and produced correct recognition results for these digits.

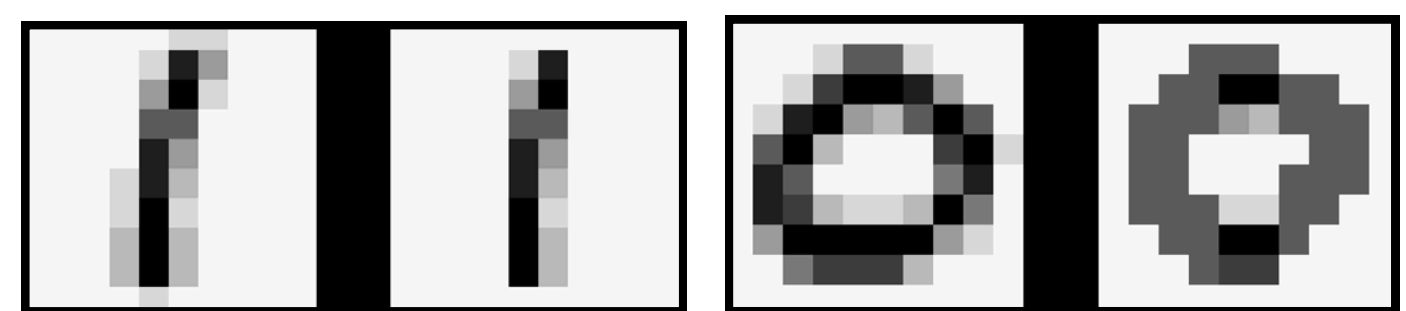


fig 6: output images

Conclusion and Future work

- Successfully performed digit recognition for custom-designed images of all 10 digits.
- Demonstrated a use case for recognizing digits in special fonts or styles.
- Achieved recognition of constrained handwritten digits (0 and 1) from the MNIST dataset.
- Future work can extend the system to handle more handwritten digits beyond 0 and 1.
- This approach can also be adapted for letter and character recognition tasks.

Neurons	Synapses	LUTs	Flip-Flops
60	3,600	6,931	2,563
100	10,000	31,114	4,792

References

- [1] M. Abernot et al., "Digital Implementation of Oscillatory Neural Network for Image Recognition Applications," Frontiers in Neuroscience, vol. 15, Aug. 2021, doi: <https://doi.org/10.3389/fnins.2021.713054>.
- [2] E. Luhulima, M. Abernot, F. Corradi, and A. Todri-Sanial, "Digital Implementation of On-Chip Hebbian Learning for Oscillatory Neural Network," pp. 1–6, Aug. 2023, doi: <https://doi.org/10.1109/islped58423.2023.10244501>.